

UNIVERSITÉ HASSAN II DE CASABLANCA
FACULTÉ DES SCIENCES BEN M'SIK
Module MLOps & DataOps

PROJET

Détection des maladies des feuilles de plantes

Conception et déploiement d'un pipeline MLOps de bout en bout, du dataset brut au modèle en production

Réalisé par :

Hasnae EL MIR	Product Owner
Hiba OUAFI	Scrum Master
Khansaa BALAKRAFAS	Data Engineer — Ingestion
Salma ZAMAKHCHARI	Data Engineer — Qualité
Ibtissam ESSADIKI	Orchestration
Chaimaa AFESS	ML Engineer
Oumaima TALBI	Déploiement
Siham SALHI	Data Analyst — EDA, Monitoring, Documentation

Encadrant : Pr. Mohammed AITDAOUD

Année universitaire 2025 – 2026

Résumé

Ce rapport présente la conception et la mise en œuvre d'un pipeline complet de Machine Learning Operations (MLOps) et Data Operations (DataOps) appliqué à la détection automatique des maladies des feuilles de plantes à partir d'images. Le projet s'appuie sur le dataset public PlantVillage, comprenant plus de 20 000 images réparties en 15 classes (combinaisons de plantes et de maladies).

La chaîne de traitement couvre l'ensemble du cycle de vie d'un système de Machine Learning en production : l'ingestion automatisée des données brutes (dlt, DuckDB), le prétraitement et le contrôle de qualité (redimensionnement, augmentation, Great Expectations), l'orchestration des pipelines (Dagster), l'entraînement et le suivi des modèles (MLflow, TensorFlow/Keras), le déploiement d'une API de prédiction (FastAPI, Docker), ainsi que la surveillance et la détection de dérive du modèle en production (monitoring applicatif, tests statistiques).

Le système final a été déployé et validé sur une infrastructure serveur partagée (plateforme Komodo), avec une API accessible publiquement, un registre de modèles opérationnel et un tableau de bord de supervision. Ce rapport détaille la contribution de chacun des huit membres de l'équipe, les choix techniques retenus, les difficultés rencontrées lors de l'intégration des différents modules, ainsi que les solutions apportées.

Mots-clés : MLOps, DataOps, Machine Learning, Deep Learning, classification d'images, PlantVillage, Dagster, MLflow, FastAPI, Docker, monitoring, détection de dérive.

Abstract

This report presents the design and implementation of a complete Machine Learning Operations (MLOps) and Data Operations (DataOps) pipeline applied to the automatic detection of plant leaf diseases from images. The project relies on the public PlantVillage dataset, comprising more than 20,000 images distributed across 15 classes (plant/disease combinations).

The pipeline covers the entire lifecycle of a production Machine Learning system: automated raw data ingestion (dlt, DuckDB), preprocessing and quality control (resizing, augmentation, Great Expectations), pipeline orchestration (Dagster), model training and tracking (MLflow, TensorFlow/Keras), deployment of a prediction API (FastAPI, Docker), as well as production monitoring and drift detection (application monitoring, statistical tests).

The final system was deployed and validated on a shared server infrastructure (Komodo platform), with a publicly accessible API, an operational model registry, and a monitoring dashboard. This report details the contribution of each of the eight team members, the technical choices made, the

difficulties encountered during the integration of the different modules, and the solutions implemented.

Keywords: MLOps, DataOps, Machine Learning, Deep Learning, image classification, PlantVillage, Dagster, MLflow, FastAPI, Docker, monitoring, drift detection.

Table des matières

Résumé	2
Abstract	2
Table des matières	4
Introduction générale.....	6
Objectifs du projet.....	6
Organisation du rapport.....	6
Chapitre 1 — Contexte, problématique et stratégie de données	8
1.1 Problématique et objectifs métier	8
1.2 Utilisateurs cibles et valeur métier	8
1.3 Stratégie de données (Data Strategy)	8
1.4 Organisation du backlog produit	9
Chapitre 2 — Organisation Agile du projet	10
2.1 Répartition des rôles.....	10
2.2 Sprints, cérémonies et suivi.....	10
2.3 Règles de contribution et gestion de version.....	10
2.4 Tests transverses du projet	10
Chapitre 3 — Ingestion des données.....	12
3.1 Source des données et méthode de téléchargement.....	12
3.2 Pipeline d'ingestion (dlt et DuckDB)	12
3.3 Simulation d'arrivée incrémentale de données	13
Outils utilisés.....	13
Chapitre 4 — Prétraitement et qualité des données	14
4.1 Pipeline de prétraitement.....	14
4.2 Augmentation des données.....	14
4.3 Détection des doublons entre splits	14
4.4 Tests et contrôle qualité.....	15
Résultats des contrôles	15
4.5 Data Contract.....	16
Chapitre 5 — Orchestration des pipelines.....	17
5.1 Choix technique et architecture des assets	17
asset_ingestion	17
asset_preprocessing	17
asset_train_model.....	17
5.2 Planification et gestion des échecs	17
5.3 Démonstration de bout en bout et limitations observées.....	18
Outils et technologies utilisés.....	19
Chapitre 6 — Modélisation et suivi des expériences (MLOps)	20

6.1 Approches de modélisation comparées	20
6.2 Gestion du déséquilibre des classes	20
6.3 Protocole d'évaluation et de suivi (MLflow).....	20
6.4 Model Registry et politique de non-régression	21
Outils utilisés.....	21
Chapitre 7 — Déploiement	23
7.1 API de prédiction (FastAPI).....	23
7.2 Conteneurisation (Docker)	23
7.3 Intégration continue (CI/CD)	24
7.4 Déploiement sur l'infrastructure partagée (Komodo).....	24
Outils utilisés.....	26
Chapitre 8 — Analyse exploratoire, monitoring et détection de dérive.....	27
8.1 Analyse exploratoire du dataset (EDA)	27
8.2 Monitoring applicatif de l'API en production	28
Architecture retenue	29
Structure des données enregistrées.....	30
Endpoint GET /metrics.....	30
Validation en environnement de production	30
8.3 Détection de dérive des données (Data Drift)	31
Méthodologie	31
Résultats de validation du script.....	32
Règle d'interprétation retenue.....	32
Point technique — sensibilité au système d'exploitation	32
8.4 Tests automatisés du module de monitoring et de dérive	33
8.5 Architecture globale du système	33
8.6 Pistes d'amélioration identifiées	35
Conclusion générale et perspectives.....	36
9.1 Synthèse du travail réalisé.....	36
9.2 Difficultés rencontrées	36
9.3 Perspectives d'amélioration	37
Annexes	38
Annexe A — Liens du projet	38
Annexe B — Glossaire technique	38

Introduction générale

Les maladies des cultures constituent l'une des principales causes de pertes agricoles à l'échelle mondiale. Selon plusieurs estimations du secteur agronomique, une part significative des récoltes est perdue chaque année faute de diagnostic précoce et fiable, en particulier dans les zones rurales où l'accès à une expertise phytosanitaire reste limité. Le développement de solutions numériques de diagnostic automatique, accessibles depuis un simple appareil photo, représente donc un enjeu à la fois technologique et sociétal important.

Le présent rapport s'inscrit dans le cadre du module MLOps & DataOps et rend compte de la réalisation d'un projet intégrateur mené par une équipe de huit étudiants, chacun responsable d'un rôle spécifique dans la chaîne de valeur d'un système de Machine Learning en production. L'objectif du projet est de concevoir, développer et déployer un système complet capable de détecter automatiquement la maladie affectant une feuille de plante à partir d'une photographie, en s'appuyant sur les meilleures pratiques MLOps : automatisation, reproductibilité, traçabilité, tests de qualité et supervision continue.

Le projet repose sur le dataset public PlantVillage (Kaggle), largement utilisé dans la littérature scientifique pour les tâches de classification de maladies végétales, et couvre trois cultures : la tomate, le poivron et la pomme de terre.

Objectifs du projet

- Mettre en place un pipeline d'ingestion et de prétraitement de données reproductible et automatisé.
- Entraîner, comparer et enregistrer plusieurs modèles de classification d'images (CNN et transfer learning) avec un suivi rigoureux des expériences.
- Orchestrer l'ensemble du pipeline (ingestion → prétraitement → entraînement) avec un outil dédié.
- Développer une API de prédiction robuste, testée, conteneurisée et déployée.
- Mettre en œuvre un système de supervision (monitoring) et de détection de dérive des données en production.
- Documenter l'ensemble de la démarche selon les standards d'un projet professionnel.

Organisation du rapport

Ce rapport est structuré en neuf chapitres correspondant aux huit rôles de l'équipe, encadrés par une introduction et une conclusion générale. Le chapitre 1 présente le contexte, la problématique et la stratégie de données. Le chapitre 2 détaille l'organisation Agile du projet. Les chapitres 3 à 7 couvrent respectivement l'ingestion des données, le prétraitement et la qualité, l'orchestration, la modélisation et le déploiement. Le chapitre 8 est consacré à l'analyse exploratoire, au monitoring

applicatif et à la détection de dérive, avant une conclusion générale reprenant les difficultés rencontrées et les perspectives d'amélioration.

Chapitre 1 — Contexte, problématique et stratégie de données

1.1 Problématique et objectifs métier

Les maladies foliaires représentent une menace constante pour les rendements agricoles. Un diagnostic tardif ou erroné conduit fréquemment à un usage excessif ou inadapté de traitements phytosanitaires, avec des conséquences économiques et environnementales négatives. La problématique centrale du projet peut être formulée ainsi : comment permettre à un agriculteur, sans expertise phytopathologique particulière, d'obtenir un diagnostic fiable et rapide à partir d'une simple photographie de feuille ?

Le projet vise à répondre à cette problématique en développant un système de classification automatique capable d'identifier, à partir d'une image, la plante concernée, la maladie éventuellement présente (ou l'état sain), ainsi qu'un score de confiance associé à la prédiction.

1.2 Utilisateurs cibles et valeur métier

Le système s'adresse principalement à trois catégories d'utilisateurs : les agriculteurs individuels, à la recherche d'un diagnostic rapide sur le terrain ; les agronomes et techniciens agricoles, qui peuvent utiliser l'outil comme aide à la décision lors de leurs visites ; et les coopératives agricoles, susceptibles d'intégrer la solution dans un service de conseil à plus grande échelle.

La valeur métier du projet réside dans la réduction du délai de diagnostic, la démocratisation de l'accès à une expertise phytosanitaire, et la possibilité de collecter, au fil du temps, des données réelles de terrain permettant d'améliorer continuellement le modèle (boucle de rétroaction propre à une démarche MLOps).

1.3 Stratégie de données (Data Strategy)

Le projet s'appuie sur le dataset public PlantVillage, disponible sur la plateforme Kaggle. Ce jeu de données est largement reconnu dans la communauté scientifique pour les tâches de classification de maladies végétales et présente les caractéristiques suivantes :

- Environ 20 638 images de feuilles, converties par la suite à un volume supérieur après augmentation ciblée (voir chapitre 4).
- 15 classes représentant des combinaisons de plante et de maladie (ou état sain), réparties sur trois cultures : tomate, poivron et pomme de terre.
- Images prises en conditions relativement contrôlées (arrière-plan neutre, éclairage homogène), ce qui constitue une limite à anticiper lors du déploiement en conditions réelles de terrain (voir chapitre 8, section sur la détection de dérive).
- Licence libre, permettant une utilisation académique et la redistribution du code associé.

Cette stratégie de données a orienté plusieurs décisions techniques prises par l'équipe, notamment la nécessité de mettre en place une étape de correction du déséquilibre entre classes (chapitre 4) et un mécanisme de détection de dérive entre les données d'entraînement et les données réellement reçues en production (chapitre 8).

1.4 Organisation du backlog produit

La vision du produit, les user stories (au nombre d'une vingtaine, couvrant les parcours utilisateur principaux) ainsi que le backlog produit ont été rédigés et priorisés dans un tableau Kanban dédié, permettant de suivre l'avancement du projet sur l'ensemble des sprints. Un exemple représentatif de user story rédigée est le suivant : « En tant qu'agriculteur, je veux envoyer une photo de feuille pour connaître la maladie détectée et son niveau de confiance, afin de décider rapidement d'un traitement adapté. »

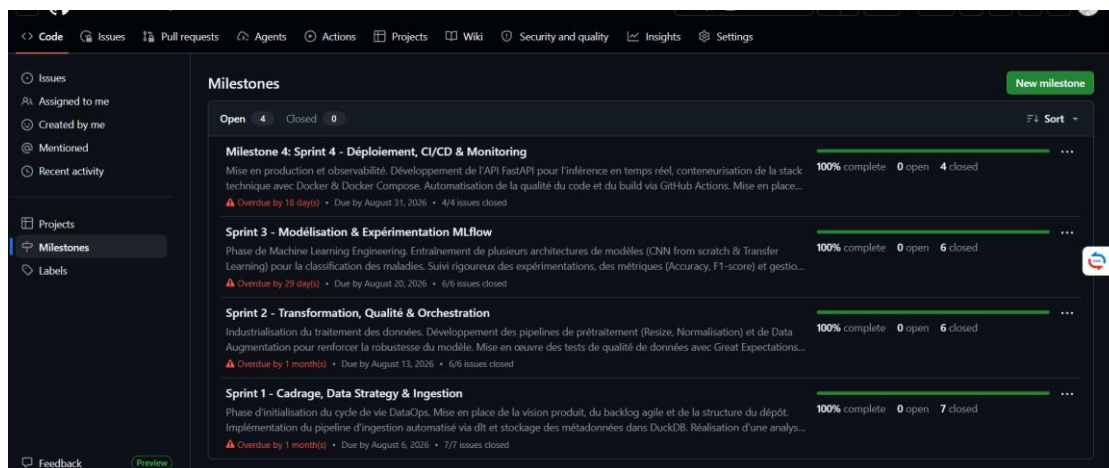


Figure 1.1 — Extrait du backlog produit (GitHub Projects)

Chapitre 2 — Organisation Agile du projet

2.1 Répartition des rôles

Le projet a été mené selon une organisation Agile inspirée de la méthode Scrum, adaptée à un contexte académique de huit membres, chacun responsable d'un périmètre technique précis et complémentaire : Product Owner, Scrum Master, deux Data Engineers (ingestion et qualité), un responsable Orchestration, un ML Engineer, un responsable Déploiement et une Data Analyst (monitoring et documentation).

Cette répartition a permis un développement en parallèle des différents modules, tout en imposant une coordination rigoureuse aux points de jonction du pipeline (voir chapitre 9, section sur les difficultés d'intégration).

2.2 Sprints, cérémonies et suivi

Le projet a été structuré en trois sprints successifs, chacun ponctué d'un Sprint Planning, d'une Sprint Review et d'une Rétrospective. Le tableau Kanban (GitHub Projects) a servi de support unique de suivi de l'avancement, garantissant une visibilité partagée sur l'état de chaque tâche.

Sprint	Contenu principal
Sprint 1	Cadrage, Data Strategy, organisation Agile, pipeline d'ingestion, analyse exploratoire (EDA)
Sprint 2	Transformation des données, contrôle qualité, tests automatisés, orchestration
Sprint 3	Modélisation & Expérimentation MLflow
Sprint 4	Déploiement, CI/CD & Monitoring

2.3 Règles de contribution et gestion de version

Les règles de collaboration Git ont été formalisées dans un fichier CONTRIBUTING.md, précisant la convention de nommage des branches (préfixe feature/ suivi du nom du module), le format attendu des messages de commit, ainsi que l'obligation de passer par une Pull Request revue avant toute fusion dans la branche principale (main).

Cette discipline s'est avérée déterminante lors de la phase d'intégration finale du pipeline (chapitre 9), où plusieurs corrections ont dû être proposées et validées collectivement avant le déploiement en production.

2.4 Tests transverses du projet

Au-delà des tests spécifiques à chaque module (détaillés dans les chapitres correspondants), une suite de tests transverses a été mise en place avec pytest, couvrant notamment la validation des

fonctions de prétraitement d'image, la vérification du bon chargement du modèle et le contrôle du comportement des endpoints critiques de l'API (/predict, /health).

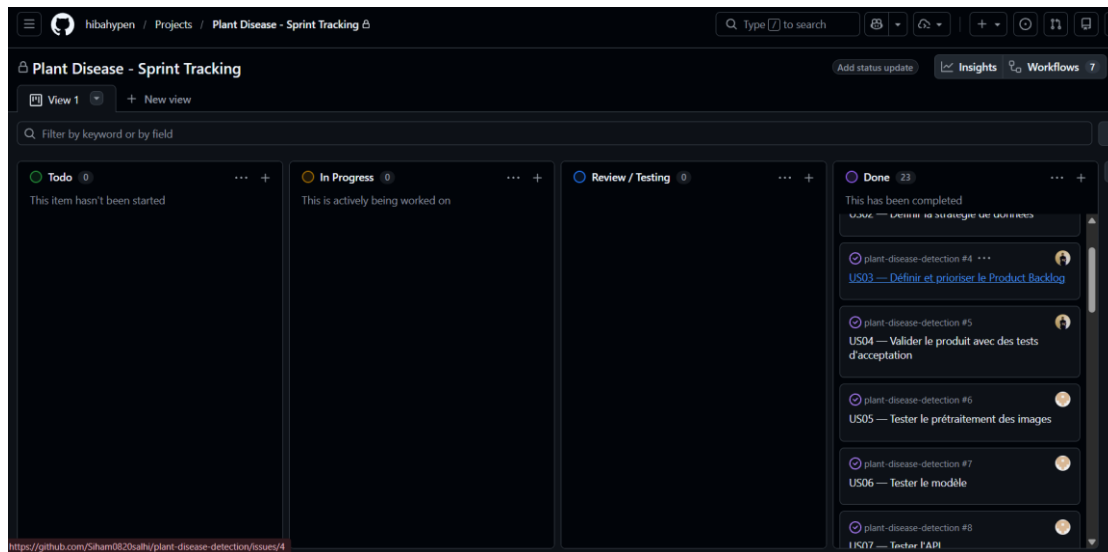


Figure 2.1 — Tableau Kanban et historique des sprints

Chapitre 3 — Ingestion des données

3.1 Source des données et méthode de téléchargement

Le dataset utilisé est PlantVillage, disponible publiquement sur Kaggle (kaggle.com/datasets/emmarex/plantdisease). Il contient 20 638 images de feuilles réparties en 15 classes, couvrant trois cultures : poivron (Pepper bell), pomme de terre (Potato) et tomate (Tomato).

Classe (plante — maladie)	Nombre d'images
Tomato — Tomato Yellow Leaf Curl Virus	3 209
Tomato — Bacterial spot	2 127
Tomato — Late blight	1 909
Tomato — Septoria leaf spot	1 771
Tomato — Spider mites (two-spotted)	1 676
Tomato — healthy	1 591
Pepper bell — healthy	1 478
Tomato — Target Spot	1 404
Tomato — Early blight	1 000
Potato — Early blight	1 000
Potato — Late blight	1 000
Pepper bell — Bacterial spot	997
Tomato — Leaf Mold	952
Tomato — Tomato mosaic virus	373
Potato — healthy	152

Le téléchargement est automatisé via l'API Kaggle (authentification par jeton personnel), garantissant que le processus est reproductible par n'importe quel membre de l'équipe sans intervention manuelle sur le site web :

```
kaggle datasets download -d emmarex/plantdisease
```

Les images sont extraites et organisées dans `data/raw/`, un dossier par classe, conformément à la convention de nommage du dataset d'origine (séparateur `__` entre le nom de la plante et celui de la maladie).

3.2 Pipeline d'ingestion (dlt et DuckDB)

Le script `pipeline_ingestion.py` constitue le cœur du module d'ingestion. Il réalise les opérations suivantes : parcours récursif du répertoire `data/raw`, extraction de la plante et de la maladie à partir du nom de chaque dossier, filtrage des fichiers non-image, génération des métadonnées (chemin, plante, maladie) et chargement automatisé de ces métadonnées dans une base DuckDB via l'outil `dlt` (data load tool), dans un schéma `raw`, table `images_metadata`.

```
pipeline = dlt.pipeline(
    pipeline_name="plantvillage_ingestion",
    destination="duckdb",
    dataset_name="raw",
)
pipeline.run(metadonnees, table_name="images_metadata", write_disposition="replace")
```

La vérification de la complétude de l'ingestion (20 638 lignes en base, une ligne par image, réparties conformément au tableau ci-dessus) a été confirmée par un script dédié, ainsi qu'un contrôle basique de lisibilité des fichiers image (bibliothèque `Pillow`) permettant d'écarter d'éventuels fichiers corrompus avant transmission à l'étape de prétraitement.

3.3 Simulation d'arrivée incrémentale de données

Afin d'anticiper l'usage réel du système — la réception progressive de photographies envoyées par des agriculteurs — et de préparer l'intégration avec l'orchestrateur (chapitre 5), le pipeline propose un second mode de chargement incrémental : le dataset est découpé en lots de 1 000 images, chaque lot étant chargé successivement dans une table dédiée (`images_metadata_incremental`) en mode `append`, garantissant qu'aucun lot précédent n'est écrasé.

Ce mode a été testé avec succès sur l'ensemble du dataset (21 lots, dont un lot partiel de 638 images), aboutissant à un total identique au chargement complet (20 638 lignes), confirmant l'absence de perte ou de duplication de données lors du chargement incrémental.

Outils utilisés

Outil	Rôle dans le pipeline
Python	Langage principal du pipeline
dlt (data load tool)	Chargement automatisé des métadonnées dans DuckDB
DuckDB	Base de données locale de stockage des métadonnées
Kaggle API	Téléchargement automatisé et reproductible du dataset
Pillow	Vérification de l'intégrité des images
Git / GitHub	Versionnement du code et documentation

Chapitre 4 — Prétraitement et qualité des données

4.1 Pipeline de prétraitement

Le prétraitement transforme les images brutes du dataset PlantVillage en données homogènes et correctement organisées pour la modélisation. Le pipeline, développé dans `preprocessing.py`, réalise successivement l'inventaire des images disponibles, leur redimensionnement uniforme à 224×224 pixels (conversion préalable au format RGB), une fonction de normalisation des pixels dans l'intervalle $[0, 1]$, puis une séparation stratifiée du dataset en trois ensembles (train, validation, test), à l'aide de `train_test_split` de Scikit-learn avec l'option `stratify` et un `random_state` fixé à 42 pour garantir la reproductibilité.

```
data/processed/  
├── train/  
├── val/  
└── test/
```

Remarque technique — la normalisation des pixels est appliquée en mémoire lors du chargement, mais les fichiers JPEG effectivement sauvegardés sur disque dans `data/processed` conservent des valeurs de pixels comprises entre 0 et 255. La normalisation doit donc être réappliquée lors du chargement des données pour l'entraînement du modèle, ce qui est bien le cas dans le pipeline de modélisation (chapitre 6).

4.2 Augmentation des données

Afin d'améliorer la diversité des données d'entraînement et de compenser le déséquilibre observé entre classes (chapitre 1, section 1.3), une étape de Data Augmentation a été mise en œuvre avec la bibliothèque `Albumentations`. Cette augmentation est appliquée exclusivement au jeu d'entraînement — les ensembles de validation et de test restent inchangés afin de préserver leur indépendance pour l'évaluation du modèle.

Le script vérifie le nombre d'images de chaque classe du jeu d'entraînement et génère de nouvelles variantes (rotation, retournement horizontal, zoom, variation de luminosité) jusqu'à atteindre un seuil minimal fixé à 1 000 images par classe lorsque cela est nécessaire, limitant ainsi le déséquilibre tout en augmentant la diversité visuelle des données.

4.3 Détection des doublons entre splits

Un contrôle spécifique a permis d'identifier des doublons résiduels du dataset d'origine, présents sous des noms de fichiers différents dans plusieurs splits distincts. Le script `detecter_doublons.py` calcule un hachage de chaque image et compare les empreintes entre les ensembles train, validation

et test ; les copies détectées dans les splits de validation et de test sont supprimées afin de préserver l'étanchéité entre les données d'entraînement et d'évaluation.

Ce contrôle a mis en évidence cinq doublons lors de l'exécution complète du pipeline, tous supprimés avec succès, ce qui souligne l'importance de ce type de vérification, y compris sur un dataset de référence largement utilisé dans la littérature.

5. Validation Great Expectations

Contrôles automatisés appliqués aux métadonnées structurées du dataset PlantVillage.

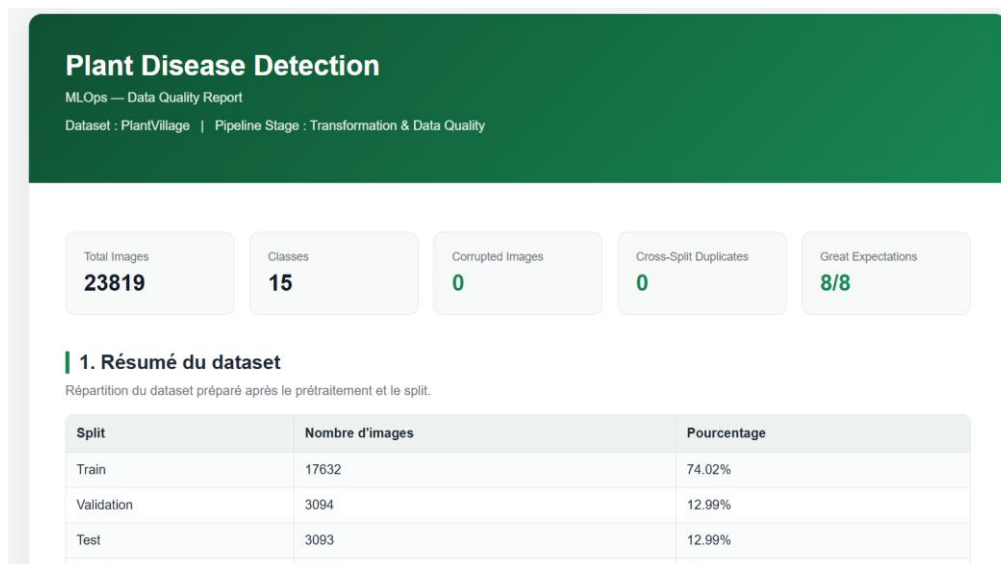
GX Tests 8	GX Passed 8	GX Failed 0	GX Global Status PASSED
Great Expectations Check		Status	
Valeurs autorisées dans split		PASSED	
Split non vide		PASSED	
Classe non vide		PASSED	
Nom du fichier non vide		PASSED	
Chemin image non vide		PASSED	
Nombre de colonnes = 4		PASSED	
Colonnes attendues		PASSED	
Nombre d'images supérieur à 0		PASSED	

4.4 Tests et contrôle qualité

Les tests automatisés (pytest) vérifient les classes présentes, les dimensions des images, la lisibilité des fichiers, le déséquilibre entre classes et l'absence de doublons entre splits. En complément, la bibliothèque Great Expectations valide automatiquement plusieurs règles de qualité sur les métadonnées du dataset (formats attendus, colonnes obligatoires, absence de valeurs vides), et génère un rapport documentant l'ensemble des vérifications effectuées.

Résultats des contrôles

1. Répartition post-prétraitement : 23 819 images après augmentation, réparties entre train (74,02 %), validation (12,99 %) et test (12,99 %).
2. Contrôles qualité des images : les 15 classes sont présentes, les images sont lisibles et de taille 224×224 , et aucun doublon ne subsiste entre les splits après nettoyage.
3. Déséquilibre des classes : la classe la moins représentée contient 1 000 images et la plus représentée 2 245 images, soit un ratio maximal/minimal de 2,25 après augmentation (contre un ratio initial d'environ 21).
4. Validation Great Expectations : 8 contrôles sur 8 réussis.
5. Statut final : dataset conforme, prêt pour l'étape d'entraînement du modèle.



2. Contrôles qualité des images

Vérification directe des fichiers après prétraitement.

Contrôle	Résultat
Nombre de classes attendu (15)	PASSED
Même classes dans train / val / test	PASSED
Images lisibles / non corrompues	PASSED 0 image(s) problématique(s)
Dimensions attendues 224 × 224	PASSED 0 erreur(s)
Absence de doublons entre les splits	PASSED 0 doublon(s)

Figure 4.1 — Rapport de qualité des données (Great Expectations)

4.5 Data Contract

Afin d'assurer la cohérence et la fiabilité des données tout au long du pipeline, un Data Contract a été formalisé dans un fichier YAML dédié, précisant les formats d'image autorisés (JPG, JPEG, PNG), les dimensions attendues (224×224 pixels), les splits autorisés, la liste des 15 classes attendues, et l'obligation d'absence d'images corrompues, de classes vides ou de doublons entre splits. Ces contraintes constituent la référence commune pour l'ensemble des contrôles automatisés du pipeline.

Chapitre 5 — Orchestration des pipelines

5.1 Choix technique et architecture des assets

L'orchestration constitue la colonne vertébrale de l'architecture MLOps du projet : elle fédère et automatise l'ensemble des modules développés individuellement par l'équipe. Dagster a été retenu comme orchestrateur, reposant sur le paradigme des Software-Defined Assets (SDA), qui matérialise chaque étape du pipeline comme un actif de données explicite, offrant une gouvernance et une traçabilité natives du lignage des données.

Le pipeline structure le projet en trois assets interconnectés par des dépendances strictes :

asset_ingestion

Encapsule l'exécution du script d'ingestion (chapitre 3) : parcours de data/raw/, extraction des métadonnées et chargement structuré dans DuckDB via dlt.

asset_preprocessing

Dépend du succès de l'ingestion. Valide la conformité du dataset avec le Data Contract, effectue le redimensionnement et réalise le partitionnement stratifié en sous-ensembles train, val et test.

asset_train_model

Dépend de la validation complète du prétraitement. Déclenche l'entraînement du modèle (chapitre 6) et assure la journalisation automatique des hyperparamètres, métriques et artefacts dans MLflow.

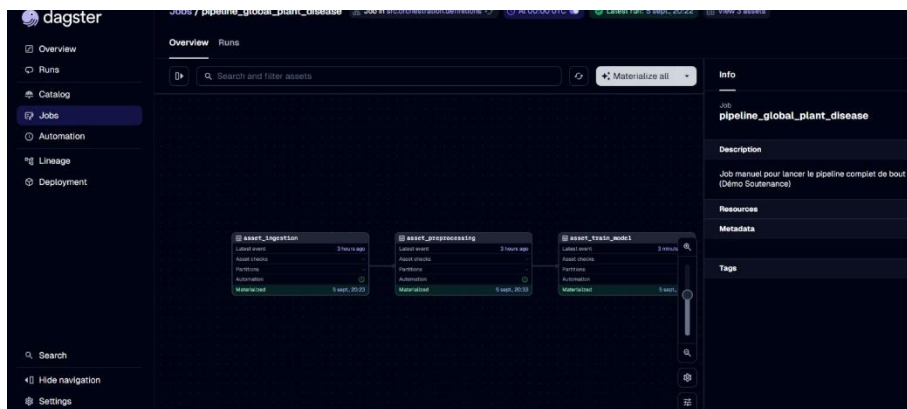


Figure 5.1 — Graphe de lignage global dans Dagster UI

5.2 Planification et gestion des échecs

Un planificateur automatique a été configuré avec l'expression cron "0 0 * * *", déclenchant chaque jour à minuit (UTC) l'exécution de l'ingestion incrémentale, la validation qualité des nouveaux arrivages et l'actualisation des données pour le modèle — simulant la réception continue

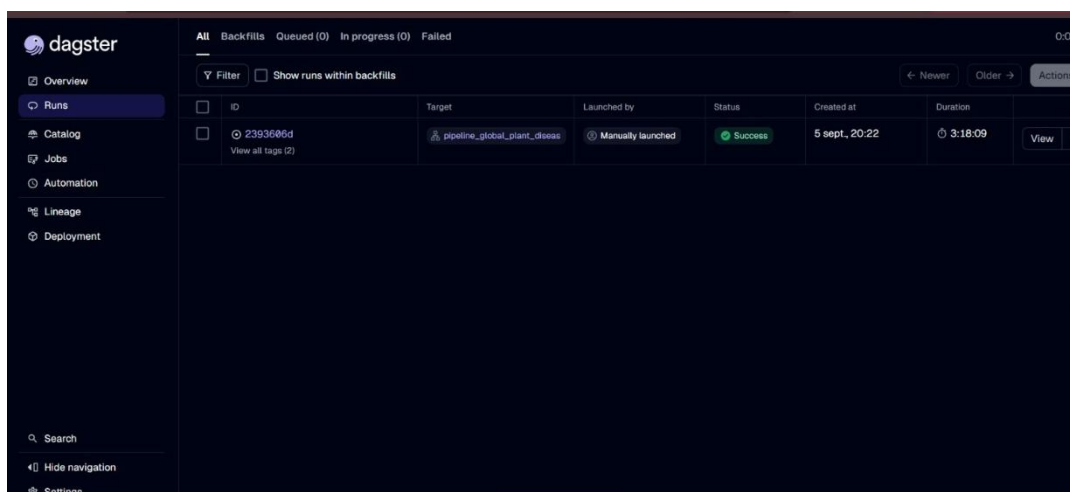
de nouvelles photographies d'agriculteurs. Un job manuel complémentaire (pipeline_global_plant_disease) permet de déclencher l'intégralité du flux de bout en bout depuis le tableau de bord, notamment à des fins de démonstration.

La politique de tolérance aux pannes repose sur le principe du fail-fast : si une anomalie survient à une étape (indisponibilité de la base lors de l'ingestion, violation du Data Contract lors du prétraitement), Dagster interrompt immédiatement l'exécution et neutralise les étapes en aval, garantissant qu'aucun modèle n'est entraîné sur des données invalides.

5.3 Démonstration de bout en bout et limitations observées

La validation empirique du système d'orchestration a été réalisée sur l'intégralité du dataset officiel. L'exécution complète du job global a permis de matérialiser avec succès les trois assets, avec les durées suivantes constatées lors d'un run représentatif : ingestion (environ deux minutes), prétraitement (environ dix minutes), entraînement du modèle (durée variable selon le modèle et le nombre d'époques configurés, pouvant atteindre plusieurs heures sur un poste de travail dépourvu de carte graphique dédiée).

***Limitation identifiée** — sous Windows, Dagster désactive par défaut la capture des journaux d'exécution ("compute log capture disabled"), ce qui a rendu difficile le suivi en temps réel de la progression de l'étape d'entraînement lors des tests locaux. Une piste de correction consiste à définir la variable d'environnement PYTHONLEGACYWINDOWSSTDIO avant le lancement de Dagster ; cette limitation ne se reproduit pas dans l'environnement Linux du déploiement conteneurisé (chapitre 7).*



The screenshot shows the Dagster web interface. On the left is a sidebar with navigation links: Overview, Runs (selected), Catalog, Jobs, Automation, Lineage, and Deployment. The main area displays a table of runs. At the top, there are tabs for 'All', 'Backfills', 'Queued (0)', 'In progress (0)', and 'Failed'. Below the tabs, there are filters and a 'Show runs within backfills' checkbox. The table has columns for ID, Target, Launched by, Status, Created at, and Duration. One run is listed with ID 2393506d, Target pipeline_global_plant_disease, Launched by Manually launched, Status Success, Created at 5 sept., 20:22, and Duration 3:18:09. There is a 'View' button next to the run.

ID	Target	Launched by	Status	Created at	Duration
2393506d	pipeline_global_plant_disease	Manually launched	Success	5 sept., 20:22	3:18:09

Figure 6.1 — Historique d'exécution d'un run complet (Dagster Runs)

Outils et technologies utilisés

Outil / Technologie	Rôle dans le pipeline d'orchestration
Dagster	Moteur d'orchestration basé sur les Software-Defined Assets et le Data Lineage
Dagster Webserver	Interface graphique web pour visualiser le lignage et piloter les exécutions
Python (subprocess)	Exécution isolée des scripts des différents modules
Cron / ScheduleDefinition	Planification automatique de l'ingestion quotidienne
DuckDB & MLflow	Systèmes cibles de stockage suivis par les assets
Docker	Intégration du service d'orchestration dans la stack conteneurisée

6.1 Approches de modélisation comparées

Le module de modélisation a été conçu pour comparer plusieurs architectures de classification d'images, selon une logique de benchmark reproductible : un CNN simple entraîné from scratch (`baseline_cnn`), servant de référence, et trois architectures de transfer learning s'appuyant sur des réseaux pré-entraînés sur ImageNet — MobileNetV2, ResNet50 et EfficientNetB0. Le script `train.py`, appellable aussi bien en ligne de commande que depuis l'orchestrateur (chapitre 5), permet de sélectionner le modèle, le nombre d'époques et la stratégie de gestion du déséquilibre des classes.

6.2 Gestion du déséquilibre des classes

Plusieurs stratégies de correction du déséquilibre ont été implémentées et rendues comparables entre elles : l'absence de correction (`none`), la pondération des classes lors de l'entraînement (`class_weight`, calculée avec `scikit-learn` selon la méthode `balanced`), l'utilisation des images augmentées hors ligne produites par le pipeline de P4 (`augmentation_p4`), l'augmentation en ligne appliquée à la volée pendant l'entraînement (`augmentation_online`), ainsi qu'une combinaison des deux dernières approches (`both`). Une résolution automatique de stratégie a également été prévue, fondée sur le ratio de déséquilibre mesuré sur les données brutes du jeu d'entraînement.

6.3 Protocole d'évaluation et de suivi (MLflow)

Chaque combinaison modèle/stratégie est entraînée et évaluée dans un run MLflow dédié, journalisant les hyperparamètres, les métriques de test (accuracy, precision, recall, F1-score macro — retenu prioritairement en raison du déséquilibre entre classes), la matrice de confusion, les courbes d'apprentissage, ainsi que le modèle lui-même comme artefact, accompagné de sa signature d'entrée/sortie.

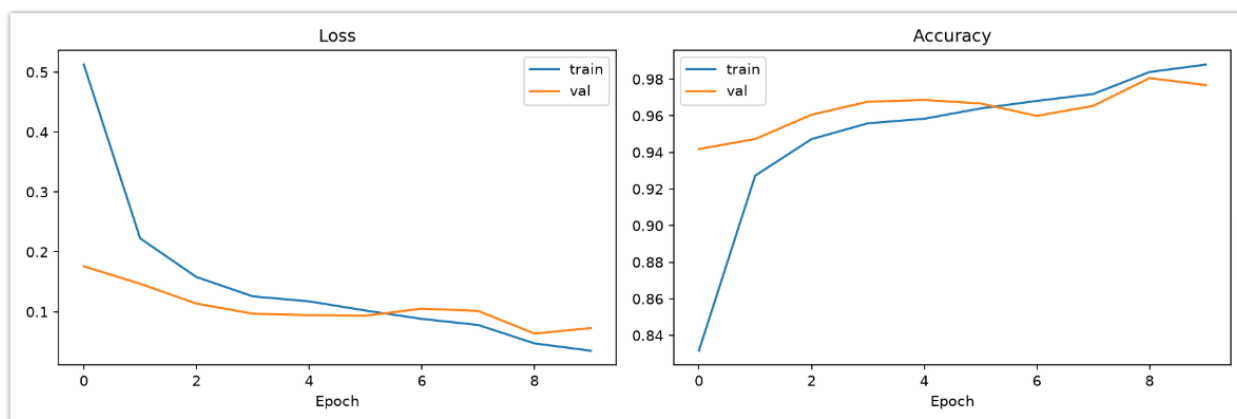


Figure 6.2 — Courbes d'apprentissage (accuracy / loss) d'un run d'entraînement

	precision	recall	f1-score	support
Pepper__bell__Bacterial_spot	0.99	1.00	1.00	149
Pepper__bell__healthy	1.00	1.00	1.00	221
Potato__Early_blight	1.00	0.99	1.00	150
Potato__Late_blight	0.97	0.97	0.97	150
Potato__healthy	0.96	1.00	0.98	23
Tomato_Bacterial_spot	0.99	0.98	0.99	319
Tomato_Early_blight	0.93	0.93	0.93	150
Tomato_Late_blight	0.98	0.95	0.97	285
Tomato_Leaf_Mold	0.97	0.98	0.97	143
Tomato_Septoria_leaf_spot	0.97	0.98	0.97	266
Tomato_Spider_mites_Two_spotted_spider_mite	0.94	0.97	0.95	252
Tomato__Target_Spot	0.95	0.92	0.93	210
Tomato__Tomato_YellowLeaf__Curl_Virus	0.99	1.00	0.99	482
Tomato__Tomato_mosaic_virus	0.98	1.00	0.99	56
Tomato_healthy	1.00	0.98	0.99	237
accuracy			0.98	3093
macro avg	0.97	0.98	0.98	3093
weighted avg	0.98	0.98	0.98	3093

Figure 6.3 — Matrice de confusion du modèle retenu en production

6.4 Model Registry et politique de non-régression

Le meilleur modèle, au sens du F1-score macro obtenu sur le jeu de test, est automatiquement enregistré dans le Model Registry de MLflow sous le nom plant-disease-classifier. Une politique de sécurité anti-downgrade a été implémentée : l'alias production n'est déplacé vers une nouvelle version que si son F1-score macro est strictement supérieur à celui du modèle actuellement en production, garantissant qu'aucune régression de performance n'est introduite involontairement lors d'un réentraînement.

Cette politique a été validée en conditions réelles : un nouveau run d'entraînement, exécuté avec un nombre d'époques volontairement réduit dans le cadre d'une démonstration de l'orchestrateur, a bien été enregistré comme nouvelle version dans le registre, sans toutefois déplacer l'alias production, celui-ci étant resté sur la version antérieure jugée plus performante — comportement conforme à la logique de non-régression implémentée.

Élément	Valeur observée
Modèle en production	ResNet50 (transfer learning)
Version enregistrée (alias production)	Version 10
Stratégie de correction du déséquilibre	augmentation_p4
Alias MLflow	production

Outils utilisés

Outil	Rôle dans le pipeline
TensorFlow / Keras	Construction et entraînement des modèles CNN et de transfer learning
MLflow (Tracking + Registry)	Suivi des expériences et gestion du cycle de vie des modèles
scikit-learn	Calcul des poids de classes, métriques d'évaluation
matplotlib	Génération des courbes d'apprentissage et matrices de confusion

Chapitre 7 — Déploiement

7.1 API de prédiction (FastAPI)

L'API de diagnostic a été développée avec FastAPI et expose trois endpoints principaux : POST /predict, qui reçoit une image de feuille au format JPG, JPEG ou PNG (limite de taille fixée à 5 Mo), la prétraite (redimensionnement à 224×224 , normalisation spécifique au modèle ResNet50) puis renvoie la plante identifiée, la maladie détectée (ou l'état sain) et le score de confiance associé ; GET /health, qui renvoie l'état de l'API et la version du modèle chargé ; GET /metrics, détaillé au chapitre 8.

La validation des entrées est assurée par Pydantic combiné à une vérification explicite du format et de la taille du fichier, renvoyant une erreur 422 claire en cas de fichier non conforme. Le modèle est chargé au démarrage de l'application (lifespan FastAPI) depuis le Model Registry MLflow, par résolution de l'alias production.

7.2 Conteneurisation (Docker)

L'ensemble du système est conteneurisé au moyen de trois services orchestrés par Docker Compose : le service api (image légère basée sur python:3.11-slim, dépendances dédiées à l'API), le service mlflow (image officielle du projet MLflow, exposant le tracking server et le model registry), et le service dagster (image incluant l'ensemble des dépendances du projet, nécessaires à l'exécution des scripts de pipeline appelés par les assets).

Chaque service dispose d'un healthcheck Docker interrogeant périodiquement son point de disponibilité, et d'une politique de redémarrage automatique en cas d'échec (restart: on-failure). Les trois services communiquent via un réseau Docker dédié, se résolvant mutuellement par leur nom de service (par exemple http://mlflow:5000 depuis le conteneur de l'API).

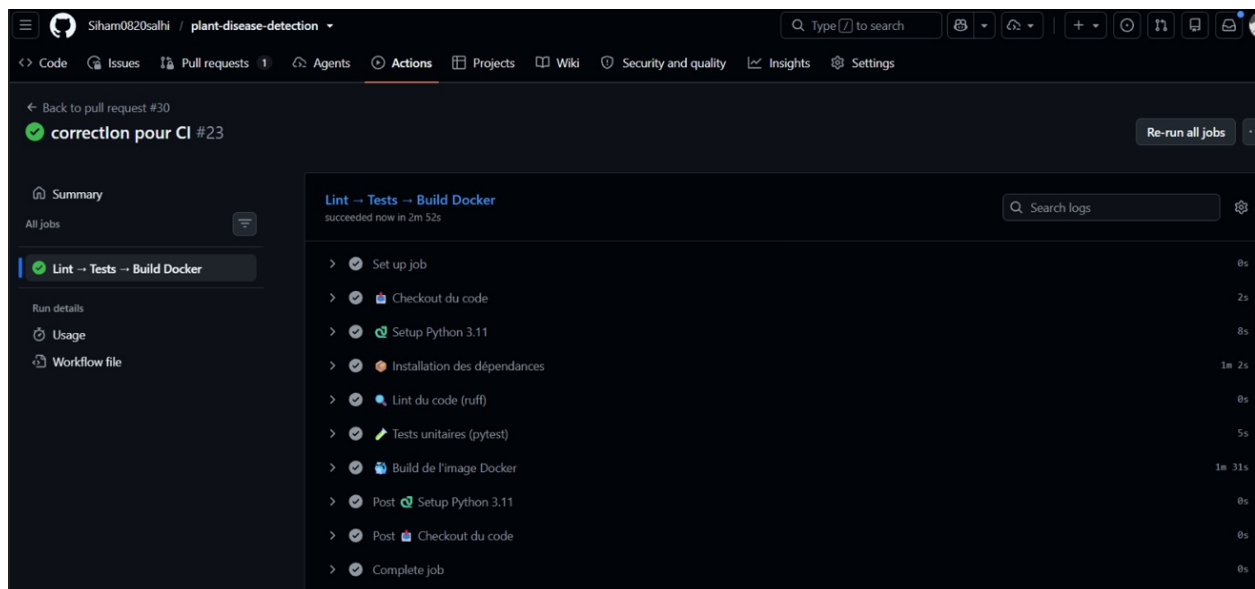


Figure 7.1 — Architecture des trois conteneurs Docker

7.3 Intégration continue (CI/CD)

Un workflow GitHub Actions dédié à l'API a été mis en place, déclenché automatiquement à chaque modification du dossier `src/api/`. Il exécute séquentiellement : la récupération du code, l'installation des dépendances légères de l'API, le contrôle de style du code avec `ruff`, l'exécution de la suite de tests `pytest` (couvrant les endpoints `/health` et `/predict`), puis la construction de l'image Docker sans publication, afin de vérifier la validité du `Dockerfile` avant toute mise en production.

Ce pipeline de CI/CD a permis de détecter et corriger plusieurs anomalies mineures de style de code (organisation des imports, gestion trop permissive d'une exception dans le module de monitoring) avant la fusion des contributions dans la branche principale.



7.4 Déploiement sur l'infrastructure partagée (Komodo)

Le déploiement final a été réalisé sur une infrastructure serveur mutualisée entre plusieurs groupes du module, pilotée par la plateforme de gestion de conteneurs Komodo. Chaque groupe dispose d'un Stack dédié (équivalent applicatif d'un fichier `docker-compose.yml`) et d'une plage de ports réservée, afin d'éviter tout conflit sur le serveur partagé.

La configuration du Stack a consisté à relier le dépôt GitHub du projet (branche main) à Komodo, en mode Git Repository, avec adaptation du mappage des ports côté hôte pour respecter la plage attribuée à l'équipe, sans modification des ports internes des conteneurs.

Service	Port interne	Port externe (plage attribuée)
api (FastAPI)	8000	3701
mlflow	5000	3702
dagster	3000	3703

La leçon méthodologique retenue de cette phase de déploiement est qu'un journal d'exécution annonçant un succès ne garantit pas, à lui seul, le bon fonctionnement réel du système : chaque étape a été confirmée par un test fonctionnel direct (appel à /health, inspection de la base de données, tentative de prédiction réelle), plutôt que par la seule lecture des journaux.

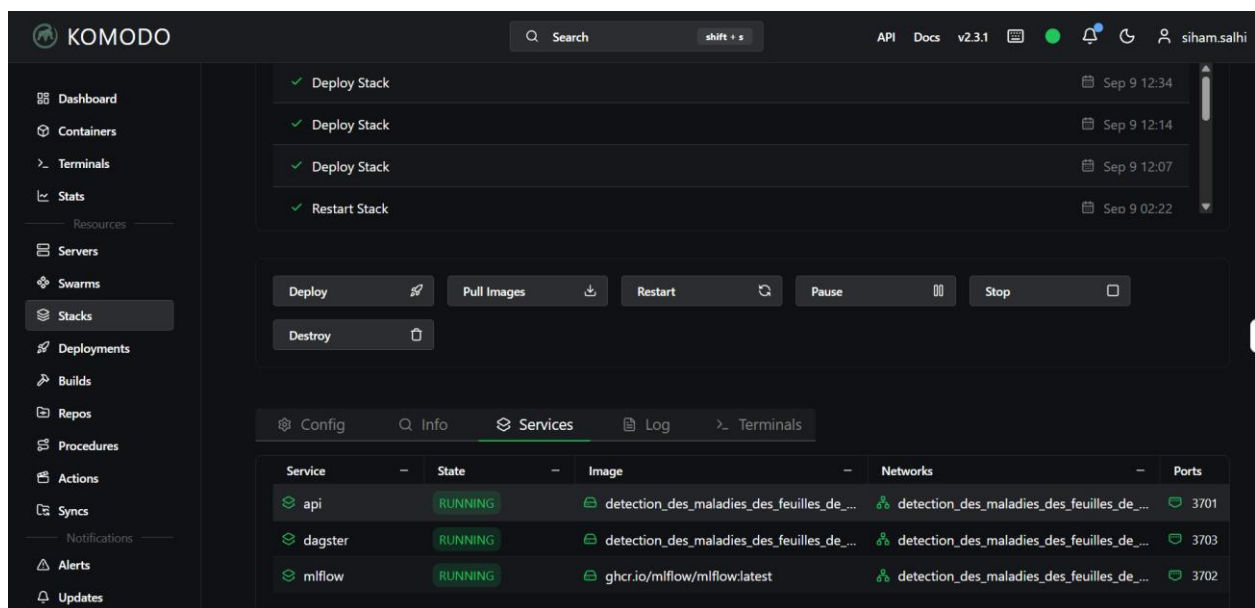


Figure 7.2 — Tableau de bord Komodo du Stack déployé

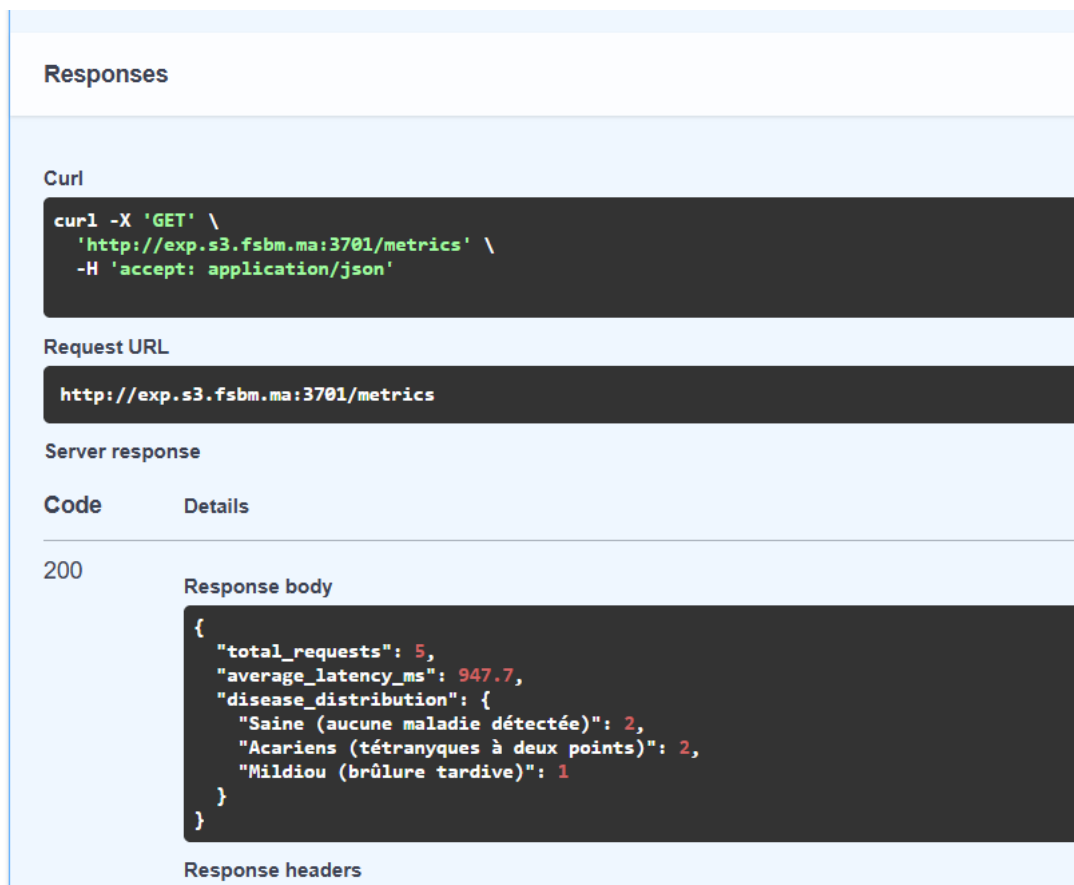


Figure 7.3 — Documentation interactive de l'API en production (Swagger)

Outils utilisés

Outil	Rôle
FastAPI / Uvicorn	Serveur applicatif de l'API
Pydantic / python-multipart	Validation des données et gestion de l'upload d'image
Docker / Docker Compose	Conteneurisation et orchestration locale des services
GitHub Actions	Intégration continue (lint, tests, build)
ruff	Analyse statique de la qualité du code
Komodo	Plateforme de déploiement sur infrastructure serveur partagée

Chapitre 8 — Analyse exploratoire, monitoring et détection de dérive

Ce chapitre couvre l'ensemble des livrables réalisés dans le cadre du rôle de Data Analyst du projet : l'analyse exploratoire du dataset en amont de la modélisation, la mise en place du monitoring applicatif de l'API en production, la détection de dérive statistique des données, ainsi que la synthèse de l'architecture globale du système.

8.1 Analyse exploratoire du dataset (EDA)

L'analyse exploratoire a été menée dès le premier sprint du projet, avant toute étape de prétraitement, afin de guider les choix techniques des équipes chargées de la transformation des données (chapitre 4) et de la modélisation (chapitre 6). Elle porte sur le volume d'images par classe, la résolution des fichiers, et l'équilibre entre classes saines et malades.

Statistique	Valeur
Nombre total d'images (avant augmentation)	20 638
Nombre de classes	15
Classe la plus représentée	Tomato — Tomato Yellow Leaf Curl Virus (3 209 images)
Classe la moins représentée	Potato — healthy (152 images)
Ratio de déséquilibre (max / min)	21,1 ×
Répartition saine / malade	15,6 % / 84,4 %

Ce déséquilibre marqué entre classes a directement motivé la stratégie d'augmentation ciblée mise en place par l'équipe en charge du prétraitement (chapitre 4, section 4.2), qui renforce les classes minoritaires jusqu'à un seuil minimal de 1 000 images, ainsi que la recommandation d'utiliser une pondération de classes (`class_weight`) lors de l'entraînement du modèle (chapitre 6).

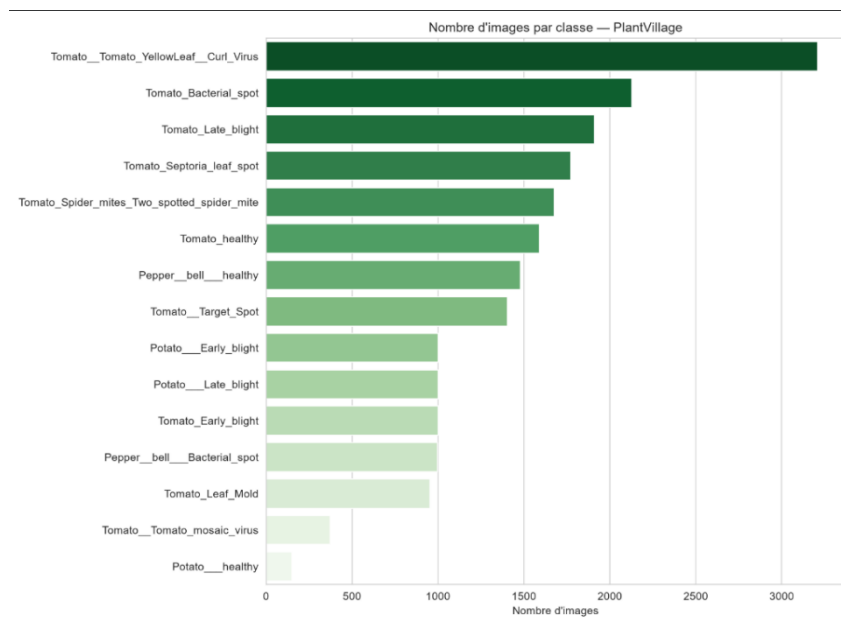


Figure 8.1 — Distribution du nombre d'images par classe

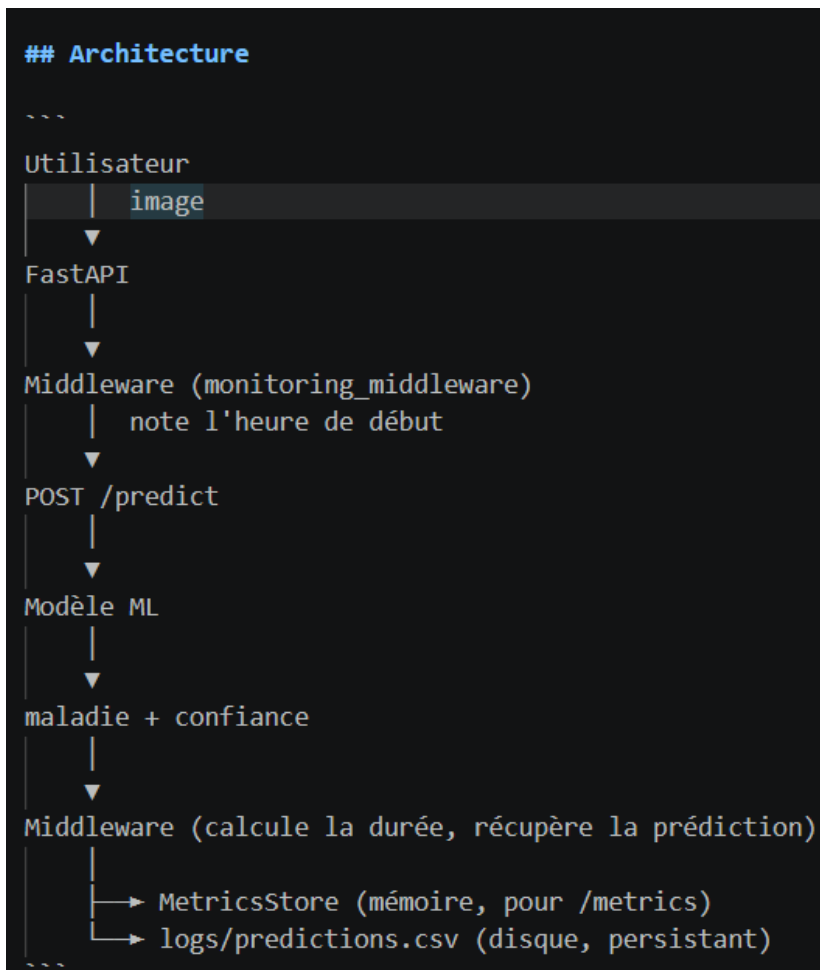
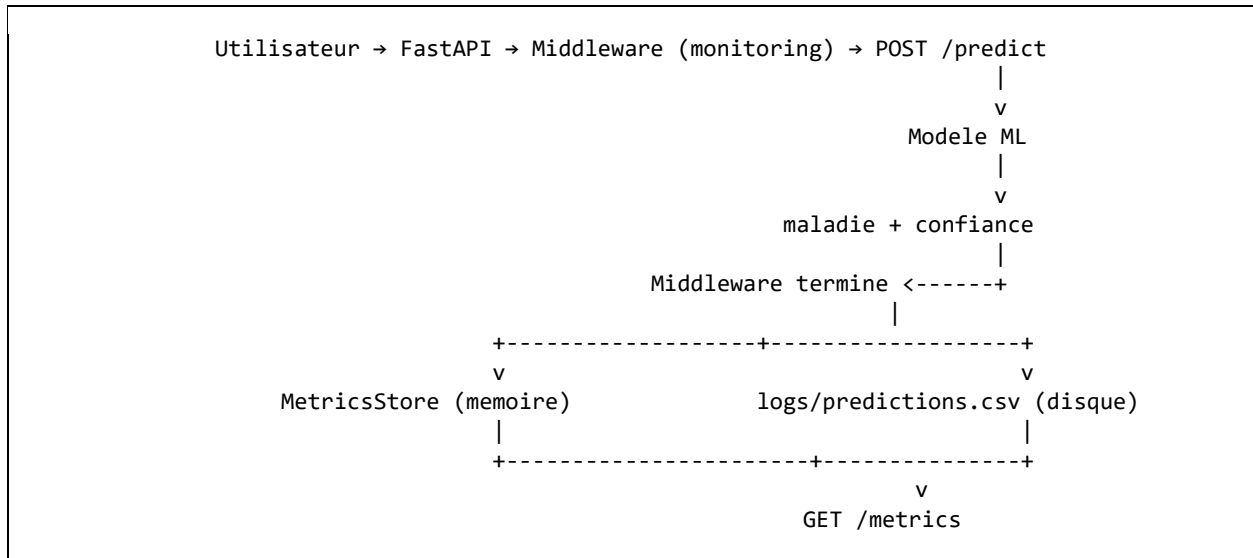


Figure 8.2 — Exemples d'images par classe et résolution moyenne

8.2 Monitoring applicatif de l'API en production

L'objectif du monitoring est de surveiller le comportement de l'API après son déploiement : à quel moment une prédiction est demandée, combien de temps elle prend à être traitée, quelle maladie est détectée et avec quel niveau de confiance.

Architecture retenue



Un middleware FastAPI mesure le temps de traitement de chaque requête et déclenche l'enregistrement pour les appels à /predict. Le stockage conserve les métriques en mémoire pour des réponses instantanées via /metrics, et les persiste simultanément dans un fichier CSV, garantissant qu'aucun historique n'est perdu en cas de redémarrage du conteneur applicatif.

Structure des données enregistrées

Champ	Description	Exemple
timestamp	Date et heure UTC de la requête (ISO 8601)	2026-09-09T13:02:25+00:00
latency_ms	Temps de traitement, en millisecondes	297,94
disease	Maladie détectée par le modèle (libellé en français)	Acariens (tétranyques à deux points)
confidence	Score de confiance de la prédiction (entre 0 et 1)	0,9998941421508789

L'écriture dans le fichier CSV est protégée par une gestion d'exception explicite : en cas d'échec de l'écriture (espace disque insuffisant, permissions), l'incident est journalisé mais la prédiction continue d'être renvoyée normalement à l'utilisateur — le monitoring ne doit en aucun cas interrompre le service rendu.

Endpoint *GET /metrics*

Cet endpoint agrège en temps réel trois statistiques d'usage : le nombre total de requêtes traitées, la latence moyenne, et la distribution des maladies prédites.

Response body

```
{
  "total_requests": 5,
  "average_latency_ms": 947.7,
  "disease_distribution": {
    "Saine (aucune maladie détectée)": 2,
    "Acariens (tétranyques à deux points)": 2,
    "Mildiou (brûlure tardive)": 1
  }
}
```

Validation en environnement de production

Le module de monitoring a été validé non seulement en environnement local, mais également sur l'infrastructure de déploiement finale (chapitre 7), à travers plusieurs prédictions réelles envoyées à l'API publiquement accessible. Les entrées suivantes, extraites directement du fichier de journalisation du conteneur en production, illustrent la diversité des classes correctement enregistrées :

```

root@286e46a2164a:/app# cat logs/predictions.csv
timestamp,latency_ms,disease,confidence
2026-09-06T21:35:43.756337+00:00,41800.58,Tache bactérienne,1.0
2026-09-06T21:35:43.791445+00:00,11.38,Tache bactérienne,1.0
2026-09-06T21:35:43.823956+00:00,9.19,Tache bactérienne,1.0
2026-09-06T21:35:43.853393+00:00,7.59,Tache bactérienne,1.0
2026-09-09T11:42:58.128516+00:00,2169.74,Saine (aucune maladie détectée),0.9999974966049194
2026-09-09T12:14:02.572513+00:00,760.06,Saine (aucune maladie détectée),0.9999973773956299
2026-09-09T13:02:25.746257+00:00,297.94,Acariens (tétranyques à deux points),0.9998941421508789
2026-09-09T13:02:27.586646+00:00,627.11,Acariens (tétranyques à deux points),0.9998941421508789

```

Figure 8.3 — Extrait du fichier de journalisation en production

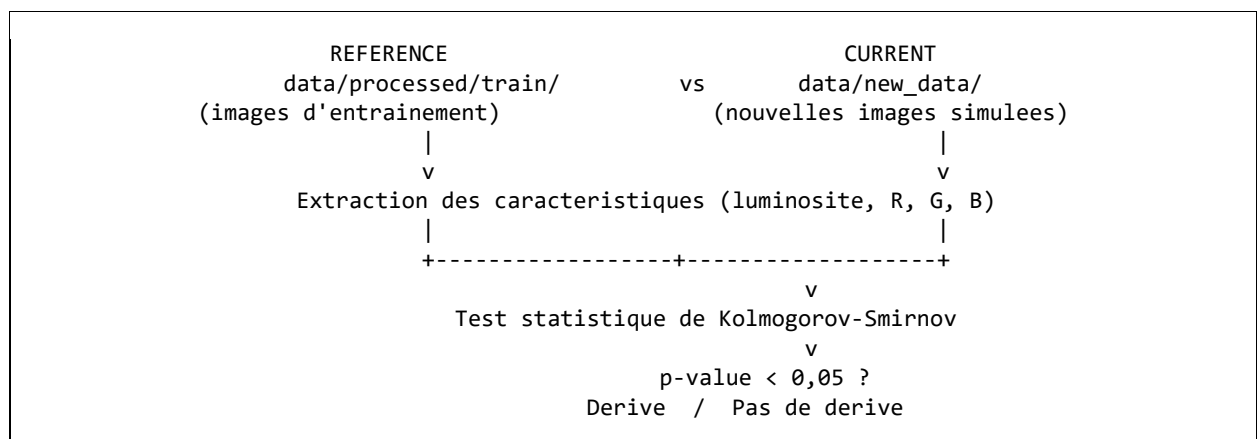
Cette validation en conditions réelles confirme que le système de monitoring fonctionne de manière identique, que l'API soit exécutée localement ou déployée sur l'infrastructure serveur partagée, ce qui constitue une garantie importante de portabilité du système.

8.3 Détection de dérive des données (Data Drift)

L'objectif de ce module est de répondre à la question suivante : les nouvelles images reçues par l'API ressemblent-elles toujours, statistiquement, aux images utilisées pour entraîner le modèle ? Le modèle a été entraîné sur des images du dataset PlantVillage, prises dans des conditions relativement contrôlées ; en production réelle, un agriculteur est susceptible d'envoyer des photographies dans des conditions très différentes (luminosité naturelle variable, arrière-plan non uniforme, angle de prise de vue, présence d'humidité sur la feuille).

Méthodologie

Le script de détection de dérive compare deux ensembles d'images selon quatre caractéristiques simples : la luminosité moyenne et les moyennes des canaux rouge, vert et bleu.



Si le dossier data/processed/train n'est pas encore disponible (pipeline de prétraitement non exécuté), le script se replie automatiquement sur data/raw — les statistiques retenues restant pertinentes, le redimensionnement n'affectant pas significativement la luminosité ou les couleurs moyennes d'une image.

Résultats de validation du script

Le test a été exécuté avec un échantillon de data/new_data provenant de la même distribution que la référence, afin de valider que le script ne génère pas de faux positifs :

Caractéristique	p-value	Moyenne référence	Moyenne nouvelle	Statut
Luminosité	0,7978	113,66	114,92	Aucune dérive
Rouge moyen	0,6898	116,54	116,95	Aucune dérive
Vert moyen	0,4042	120,37	121,77	Aucune dérive
Bleu moyen	0,9265	104,08	106,04	Aucune dérive

Aucune dérive statistiquement significative n'est détectée, résultat cohérent puisque la référence et l'échantillon actuel proviennent de la même source — ce test confirme que le script ne signale pas de dérive à tort. La validation inverse (référence et données actuelles rendues volontairement très différentes) a également été réalisée par les tests automatisés du module (section 8.4), confirmant que le script détecte correctement une dérive lorsqu'elle existe réellement.

Règle d'interprétation retenue

Une p-value inférieure à 0,05 signale une dérive statistiquement significative ; une p-value supérieure ou égale à 0,05 indique l'absence de dérive détectable. Ce seuil constitue un signal statistique à investiguer et non une preuve absolue que le modèle serait devenu défaillant — cette nuance a été retenue tout au long de l'implémentation.

Point technique — sensibilité au système d'exploitation

Un piège technique a été identifié lors du développement : le système de fichiers Windows ne distingue pas les majuscules des minuscules dans les extensions de fichiers. Une recherche successive des motifs *.jpg puis *.JPG renvoyait donc deux fois les mêmes fichiers, faussant les moyennes calculées et provoquant de faux positifs de dérive. La correction a consisté à dédupliquer les chemins collectés au moyen d'un ensemble (set) plutôt que d'une liste, rendant le script robuste indépendamment du système d'exploitation d'exécution.

```

(.venv) PS C:\Users\USER\Desktop\plant-disease-detection> python -m src.monitoring.drift
Reference trouvee dans : C:\Users\USER\Desktop\plant-disease-detection\data\processed\train
-> 500 images de reference chargees.
Chargement des nouvelles images depuis C:\Users\USER\Desktop\plant-disease-detection\data\new_data...
-> 80 nouvelles images chargees.

=== Rapport de derive (drift) ===
luminosite      | p-value=0.7107 | ref_mean=114.18 | new_mean=111.59 | ☒ OK
rouge_moyen     | p-value=0.5711 | ref_mean=116.56 | new_mean=113.26 | ☒ OK
vert_moyen      | p-value=0.6407 | ref_mean=120.70 | new_mean=118.76 | ☒ OK
bleu_moyen      | p-value=0.4042 | ref_mean=105.29 | new_mean=102.74 | ☒ OK

☒ Aucune derive significative detectee.

```

Figure 8.4 — Rapport de dérive généré par le script (exécution locale)

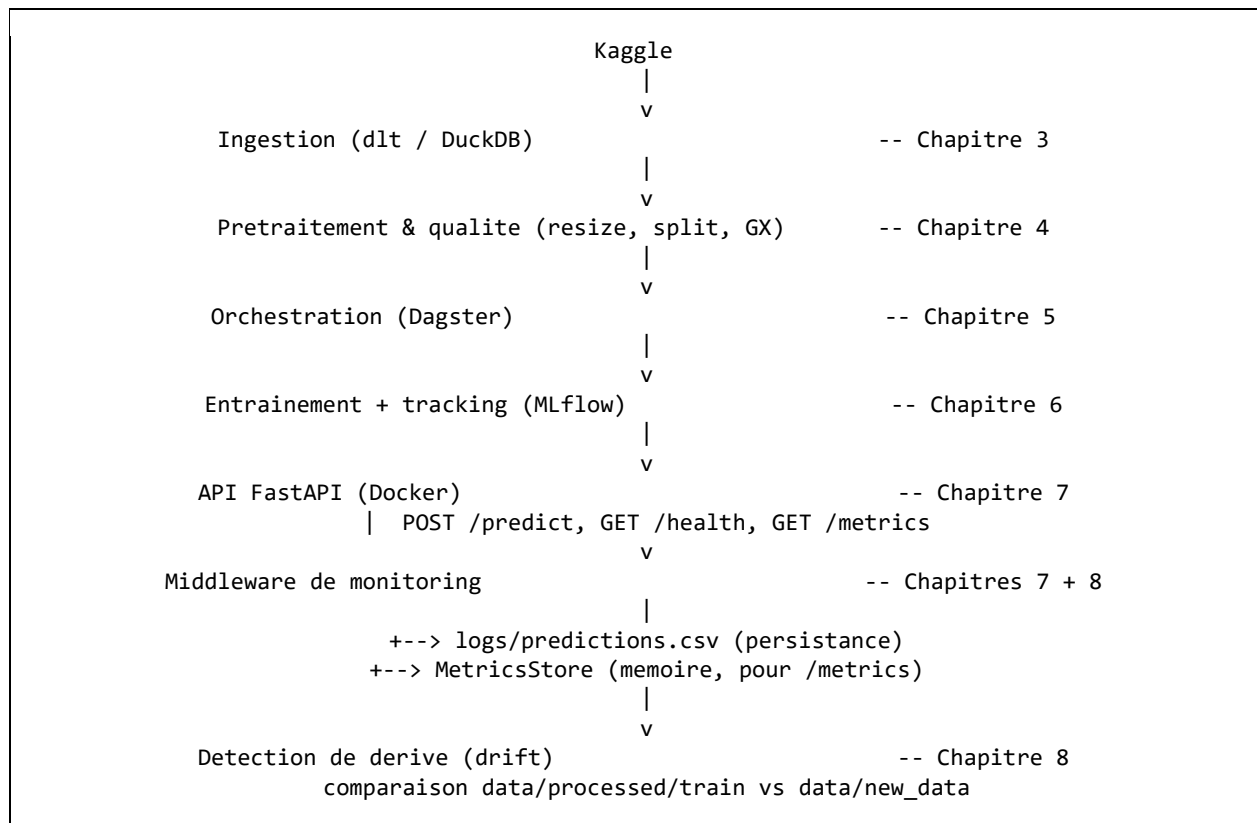
8.4 Tests automatisés du module de monitoring et de dérive

Conformément aux exigences de qualité logicielle du projet, l'ensemble du module a été couvert par une suite de tests automatisés (pytest), organisée en trois fichiers, totalisant onze tests unitaires, tous exécutés avec succès :

Fichier de test	Objet de la validation
test_monitoring.py	Enregistrement correct des prédictions en mémoire et dans le fichier CSV ; accumulation correcte de plusieurs requêtes successives
test_metrics.py	Exactitude des calculs de l'endpoint /metrics : total de requêtes, latence moyenne, distribution des maladies, comportement à vide
test_drift.py	Absence de dérive signalée lorsque référence et données actuelles sont identiques ; dérive correctement signalée lorsqu'elles sont significativement différentes

8.5 Architecture globale du système

Le schéma ci-dessous synthétise l'ensemble de la chaîne, de l'ingestion des données jusqu'à la surveillance du modèle en production, en indiquant la contribution de chaque membre de l'équipe à chaque étape.



Deux choix d'architecture méritent d'être soulignés. D'une part, le stockage des métriques combine mémoire (rapidité de réponse) et fichier CSV (persistance), compromis suffisant pour l'échelle du présent projet académique — une base de données ou un outil de type Prometheus constituerait une évolution naturelle à plus grande échelle. D'autre part, la référence utilisée pour la détection de dérive s'adapte automatiquement à l'avancement du pipeline, ce qui a permis de progresser sur cette tâche sans être bloqué par l'avancement des modules amont, conformément à la règle de non-blocage adoptée par l'équipe dès le lancement du projet.

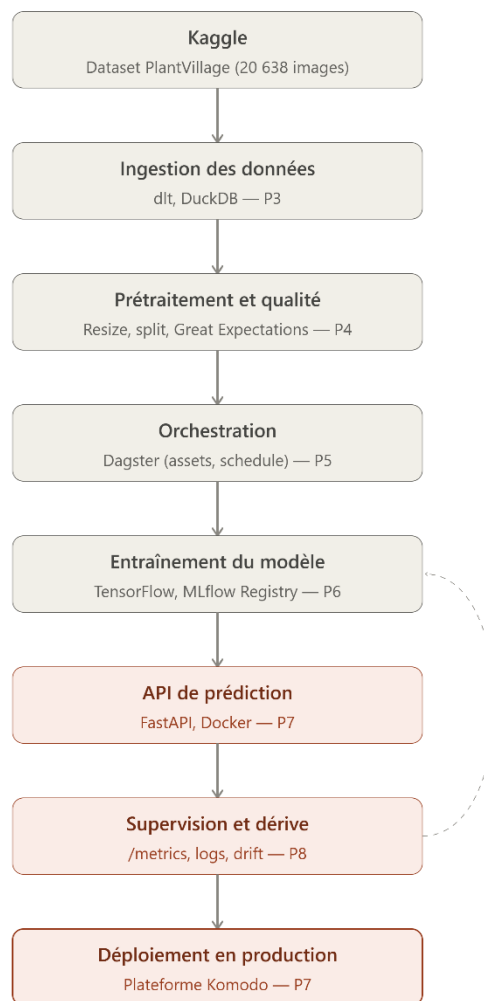


Figure 8.5 — Schéma d'architecture globale du système

8.6 Pistes d'amélioration identifiées

- Génération automatique d'un rapport HTML de dérive à l'aide de la bibliothèque Evidently, pour une visualisation enrichie lors de démonstrations futures.
- Simulation d'une dérive réaliste (par exemple un assombrissement artificiel des images de data/new_data) afin de démontrer explicitement la capacité de détection sur un cas positif construit.
- Rechargement de l'historique des métriques en mémoire depuis le fichier CSV au redémarrage de l'API, afin de ne pas perdre les compteurs cumulés lors d'un redéploiement.
- Migration du stockage des métriques vers une base de données dédiée en cas de montée en charge significative du nombre de requêtes.

Conclusion générale et perspectives

9.1 Synthèse du travail réalisé

Ce projet a permis de concevoir, développer et déployer un pipeline MLOps complet, couvrant l'ensemble du cycle de vie d'un système de Machine Learning en production : de l'ingestion automatisée de plus de 20 000 images jusqu'à une API de diagnostic accessible publiquement, en passant par le prétraitement, le contrôle de qualité, l'orchestration, l'entraînement comparatif de plusieurs modèles, le suivi des expériences, la conteneurisation et la supervision continue.

L'ensemble des huit rôles définis en début de projet a été mené à terme, avec une validation systématique de chaque composant par des tests automatisés (pytest), une intégration continue (GitHub Actions) et une démonstration de bout en bout, aussi bien en environnement local que sur l'infrastructure serveur partagée fournie pour le module.

9.2 Difficultés rencontrées

La principale difficulté rencontrée par l'équipe n'a pas résidé dans le développement isolé de chaque module — chacun ayant globalement fonctionné correctement sur la machine de son auteur — mais dans la phase d'intégration finale, lorsque l'ensemble des composants a dû être exécuté ensemble, sur une même machine puis sur un serveur distant. Cette phase a mis en évidence plusieurs catégories récurrentes de problèmes, résumées ci-dessous à des fins de retour d'expérience pour de futurs projets similaires :

6. Chemins de fichiers codés en dur et non portables, écrits en supposant une structure de dossier ou un système d'exploitation particulier (Windows chez plusieurs membres de l'équipe, Linux dans les conteneurs Docker et sur le serveur de déploiement).
7. Différences de versions de bibliothèques entre les postes de développement des membres de l'équipe, notamment MLflow, ayant provoqué des incompatibilités de schéma de base de données lors du partage de fichiers d'expérience.
8. Contraintes matérielles limitant l'entraînement de modèles profonds sur des postes de travail sans carte graphique dédiée, imposant des choix pragmatiques (réduction du nombre d'époques pour les démonstrations, réutilisation d'un modèle déjà entraîné plutôt qu'un réentraînement systématique).
9. Nécessité de transférer des artefacts volumineux (poids de modèles) non versionnés dans Git vers un environnement de déploiement distant, résolue par la publication de ces artefacts sous forme de Release GitHub téléchargeable.
10. Importance de la vérification systématique de chaque étape par un test fonctionnel réel, plutôt que par la seule lecture de journaux d'exécution annonçant un succès apparent.

Chacune de ces difficultés a été documentée et corrigée collectivement, avec un partage explicite des correctifs entre les membres concernés, conformément aux règles de contribution définies en début de projet (chapitre 2).

9.3 Perspectives d'amélioration

- Mise en place d'un stockage centralisé et partagé pour les expériences MLflow (base de données et stockage d'artefacts distants), évitant la duplication de fichiers entre les postes des membres de l'équipe.
- Harmonisation systématique des versions de dépendances entre tous les environnements (fichier requirements.txt unique et exhaustif, utilisation d'un environnement conteneurisé dès la phase de développement).
- Automatisation complète du redéploiement via un webhook déclenché à chaque fusion sur la branche principale, actuellement configuré mais non systématiquement exploité.
- Enrichissement de la détection de dérive avec des caractéristiques visuelles plus élaborées (histogrammes de couleur, caractéristiques extraites par un réseau de neurones convolutif) et génération automatique de rapports visuels avec Evidently.
- Collecte progressive de données réelles issues des utilisateurs finaux (avec leur consentement), afin d'alimenter un cycle de réentraînement périodique du modèle et de réduire le risque de dérive à long terme.

Ce projet illustre, à l'échelle d'une équipe de huit personnes, les enjeux concrets d'un déploiement MLOps réaliste : la valeur d'un tel projet ne réside pas uniquement dans la performance du modèle de Machine Learning, mais dans la robustesse, la reproductibilité et l'observabilité de l'ensemble de la chaîne qui le porte jusqu'à l'utilisateur final.

Annexes

Annexe A — Liens du projet

- Dépôt GitHub : <https://github.com/Siham0820salhi/plant-disease-detection>
- Dataset PlantVillage (Kaggle) : <https://www.kaggle.com/datasets/emmarex/plantdisease>
- API de production (Swagger) : <http://exp.s3.fsbm.ma:3701/docs>
- Interface MLflow (production) : <http://exp.s3.fsbm.ma:3702>
- Interface Dagster (production) : <http://exp.s3.fsbm.ma:3703>

Annexe B — Glossaire technique

Terme	Définition
MLOps	Ensemble de pratiques visant à automatiser et fiabiliser le cycle de vie des modèles de Machine Learning en production.
DataOps	Ensemble de pratiques appliquant les principes d'ingénierie logicielle à la gestion et à la qualité des données.
Pipeline	Enchaînement automatisé d'étapes de traitement de données ou de modèles.
Data drift (dérive des données)	Changement statistique de la distribution des données en production par rapport aux données d'entraînement.
Transfer learning	Réutilisation d'un modèle pré-entraîné sur une tâche source pour l'adapter à une nouvelle tâche cible.
Model Registry	Composant logiciel permettant de versionner, comparer et déployer des modèles de Machine Learning.
Endpoint	Point d'accès d'une API, identifié par une URL et une méthode HTTP.
Conteneurisation	Encapsulation d'une application et de ses dépendances dans un environnement isolé et portable (conteneur).